Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory Work 2: Study and Empirical Analysis of Sorting Algorithms

Author:
st. gr. FAF-242 Tacu Igor

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2025

Contents

OBJECTIVE **2**
Tasks 2

THEORETICAL BACKGROUND **2**
Empirical Analysis 2
Sorting Algorithms Overview 3

INPUT FORMAT **3**

IMPLEMENTATION **3**
QuickSort 4
MergeSort 4
HeapSort 5
BubbleSort 6

RESULTS AND ANALYSIS **6**
All Algorithms – Random Input 6
All Algorithms – Sorted Input 7
All Algorithms – Reverse-Sorted Input 8
Fast Algorithms – Larger Scale 8
Per-Algorithm – Input-Type Comparison 10

EDGE-CASE ANALYSIS **13**

CONCLUSION **14**

OBJECTIVE

Study and empirically analyze the most common comparison-based sorting algorithms, measuring their execution time across multiple input types and sizes, and identify each algorithm's strengths and limitations in practice.

Tasks

1. Implement four sorting algorithms: QuickSort, MergeSort, HeapSort, and BubbleSort.
2. Design the input format (sizes and types) for the empirical analysis.
3. Select execution time as the comparison metric.
4. Run each algorithm on three input types: random, sorted, and reverse-sorted.
5. Generate graphical representations of the results.
6. Test algorithms on edge-case inputs (empty arrays, nulls, NaN, complex numbers).
7. Deduce conclusions from the obtained data.

THEORETICAL BACKGROUND

Empirical Analysis

Empirical analysis is an experimental approach to measuring algorithm efficiency. Rather than deriving complexity through mathematical reasoning alone, the algorithm is implemented and executed on controlled input datasets, and the observed metrics (here: wall-clock execution time) are recorded and plotted. This approach is especially useful when:

- the mathematical analysis is complex or unavailable;
- comparing practical performance of algorithms with identical asymptotic complexity;
- assessing the impact of hardware and implementation details on real runtimes.

Each algorithm was run **5 times** per input size, and the **average** time (in milliseconds) was recorded to smooth out noise from OS scheduling and JVM warm-up effects.

Sorting Algorithms Overview

The four algorithms selected span a range of asymptotic complexities and practical behaviors, making them a representative sample for comparison:

Algorithm	**Best Case**	**Average Case**	**Worst Case**	**Space**
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
BubbleSort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Table 1: Theoretical time and space complexity of the four sorting algorithms.

INPUT FORMAT

Each algorithm was tested against two sets of input sizes:

- **Full set** (all 4 algorithms, including BubbleSort):
  $n \in \{250,\ 500,\ 750,\ 1000,\ 1500,\ 2000,\ 3000,\ 4000,\ 5000,\ 7500,\ 10000,\ 15000,\ 20000\}$
- **Large set** (QuickSort, MergeSort, HeapSort only):
  $n \in \{500,\ 1000,\ 2000,\ 5000,\ 10000,\ 20000,\ 40000,\ 60000,\ 80000,\ 100000\}$

For every size n, three input arrays were generated:

1. **Random** – values drawn uniformly from $[0, 100000)$ using a fixed seed.
2. **Sorted** – array $[0, 1, 2, \ldots, n-1]$; represents the best case for some algorithms.
3. **Reverse-sorted** – array $[n, n-1, \ldots, 1]$; represents a common worst case.

IMPLEMENTATION

All four algorithms were implemented in Java using a Maven project structure. Each class provides two overloads: a primitive `int[]` version for the empirical benchmarks, and a generic `T[] + Comparator<T>` version used in edge-case testing. The timing harness runs the sort 5 times on a fresh copy of the input and returns the average.

QuickSort

QuickSort partitions the array around a pivot and recursively sorts each partition. The naive last-element pivot causes $O(n^2)$ performance on sorted inputs; to avoid this, a **median-of-three** pivot selection strategy was implemented: the median of `arr[low]`, `arr[mid]`, and `arr[high]` is chosen and placed at `arr[high]` before partitioning.

Listing 1: QuickSort – median-of-three pivot and partition

```
static int medianOfThree(int[] arr, int low, int high) {
    int mid = low + (high - low) / 2;
    if (arr[low] > arr[mid])  swap(arr, low, mid);
    if (arr[low] > arr[high]) swap(arr, low, high);
    if (arr[mid] > arr[high]) swap(arr, mid, high);
    swap(arr, mid, high); // pivot -> arr[high]
    return arr[high];
}

static int partition(int[] arr, int low, int high) {
    if (high - low >= 2) medianOfThree(arr, low, high);
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot) swap(arr, ++i, j);
    swap(arr, i + 1, high);
    return i + 1;
}

static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

MergeSort

MergeSort divides the array in half, recursively sorts both halves, and merges them. Its $O(n \log n)$ worst-case guarantee makes it stable and predictable across all input types. A common bug – omitting `k++` in the final merge loop – was present in the skeleton and has been fixed.

Listing 2: MergeSort – merge step

```
static void merge(int[] arr, int l, int m, int r) {
    int[] L = Arrays.copyOfRange(arr, l, m + 1);
    int[] R = Arrays.copyOfRange(arr, m + 1, r + 1);
```

```
    int i = 0, j = 0, k = l;
    while (i < L.length && j < R.length)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < L.length) arr[k++] = L[i++];
    while (j < R.length) arr[k++] = R[j++]; // bug fix: k++ was missing
}

static void mergeSort(int[] arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
```

HeapSort

HeapSort builds a max-heap from the input array, then repeatedly extracts the maximum element to produce a sorted sequence. It achieves $O(n \log n)$ in all cases and operates in-place.

Listing 3: HeapSort – heapify and sort

```
static void heapify(int[] arr, int n, int i) {
    int largest = i, l = 2*i+1, r = 2*i+2;
    if (l < n && arr[l] > arr[largest]) largest = l;
    if (r < n && arr[r] > arr[largest]) largest = r;
    if (largest != i) {
        int tmp = arr[i]; arr[i] = arr[largest]; arr[largest] = tmp;
        heapify(arr, n, largest);
    }
}

static void heapSort(int[] arr) {
    int n = arr.length;
    for (int i = n/2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        int tmp = arr[0]; arr[0] = arr[i]; arr[i] = tmp;
        heapify(arr, i, 0);
    }
}
```

BubbleSort

BubbleSort repeatedly passes through the array, swapping adjacent elements that are out of order. An early-exit flag (`swapped`) is used so that the algorithm terminates in $O(n)$ when the input is already sorted – which is reflected clearly in the empirical results.

Listing 4: BubbleSort – with early-exit optimization

```
static void bubbleSort(int[] arr, int n) {
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int tmp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = tmp;
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted
    }
}
```

RESULTS AND ANALYSIS

All Algorithms – Random Input

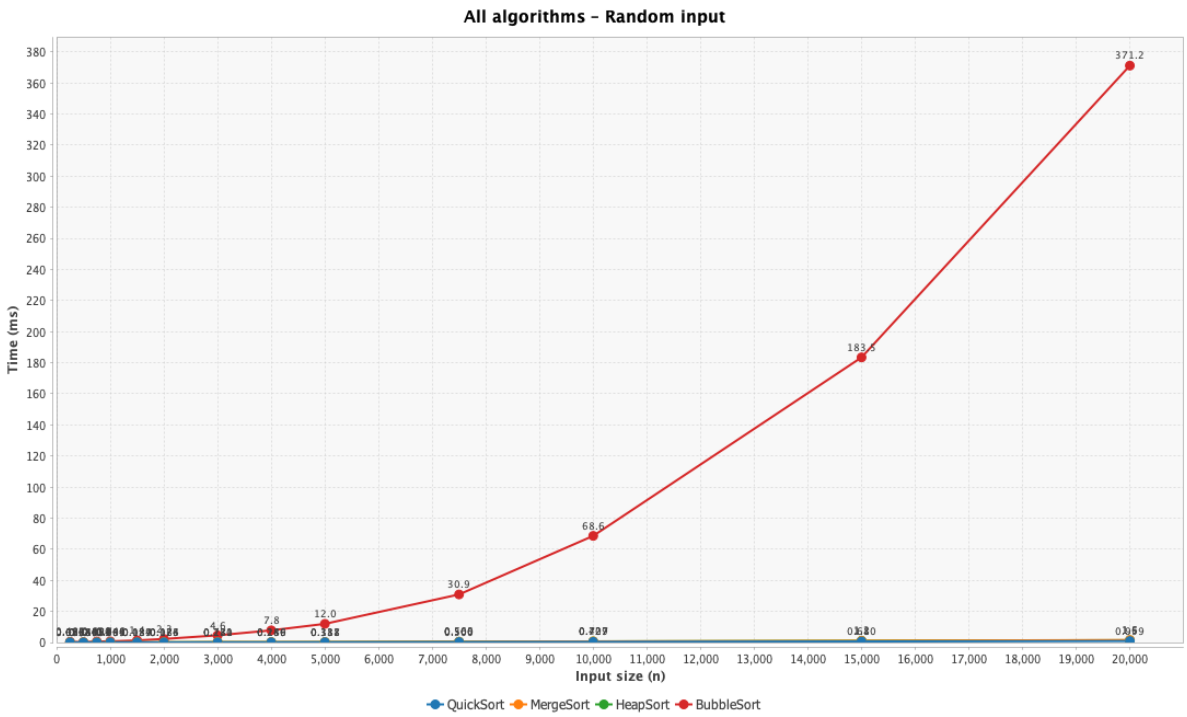

Figure 1: Execution time of all four algorithms on **random** input (n up to 20 000).

On random input (Figure 1), BubbleSort's $O(n^2)$ growth dominates the chart as expected – at $n = 20000$ it takes over **370 ms**, while the three $O(n \log n)$ algorithms remain below 2 ms. QuickSort is the fastest of the three, followed by MergeSort and HeapSort, all staying under 2 ms at $n = 20000$.

All Algorithms – Sorted Input

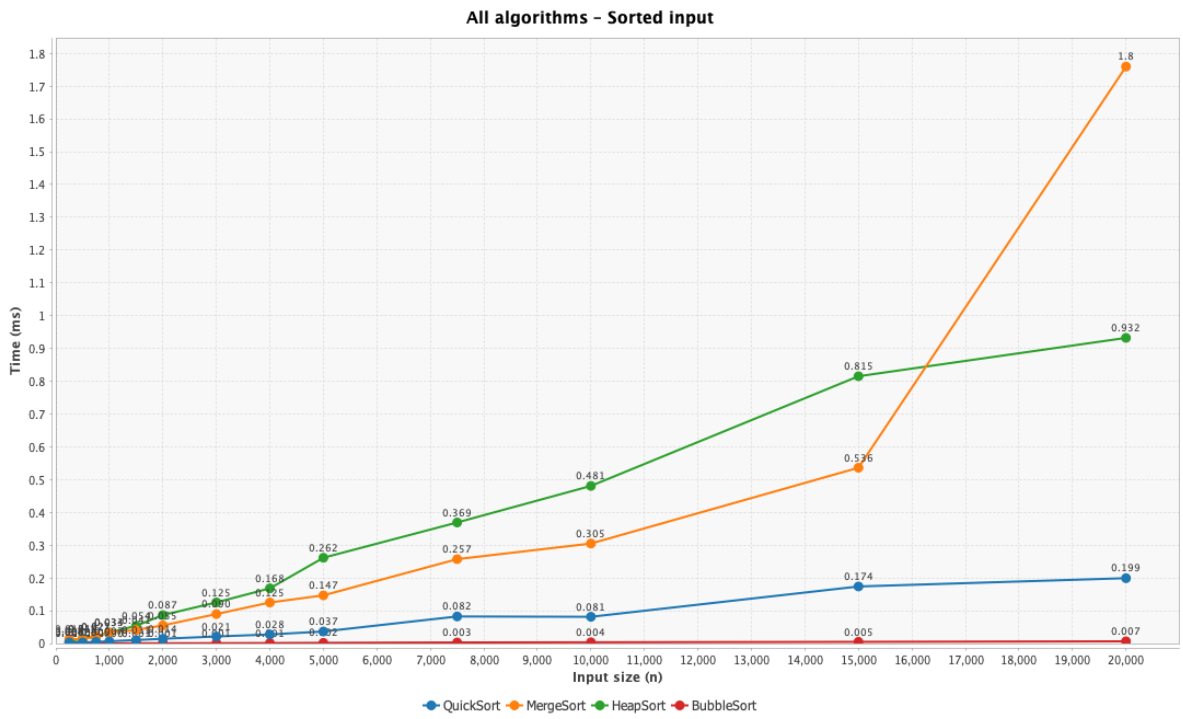

Figure 2: Execution time of all four algorithms on **sorted** input (n up to 20 000).

Sorted input (Figure 2) reveals the most interesting behavior differences. BubbleSort becomes *instant* (< 0.01 ms for any n) due to the early-exit flag. With the median-of-three pivot, QuickSort also performs well on sorted inputs. MergeSort remains consistent, while HeapSort's heap-building overhead makes it slightly slower than MergeSort on pre-sorted data.

All Algorithms – Reverse-Sorted Input

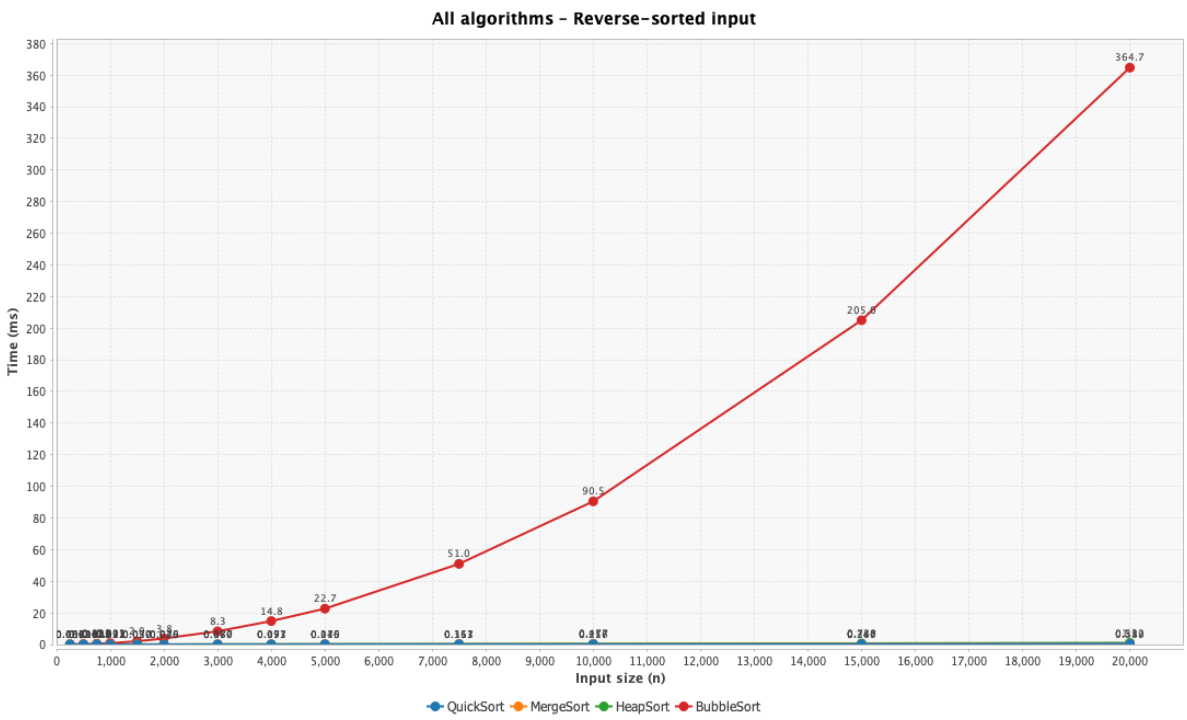

Figure 3: Execution time of all four algorithms on **reverse-sorted** input (n up to 20 000).

Reverse-sorted input (Figure 3) is a worst case for BubbleSort (**365 ms** at $n = 20000$) because every element must bubble all the way to the front. The $O(n \log n)$ algorithms are unaffected in a meaningful way.

Fast Algorithms – Larger Scale

To see meaningful differences between QuickSort, MergeSort, and HeapSort, BubbleSort was excluded and the input range was extended to $n = 100\,000$.

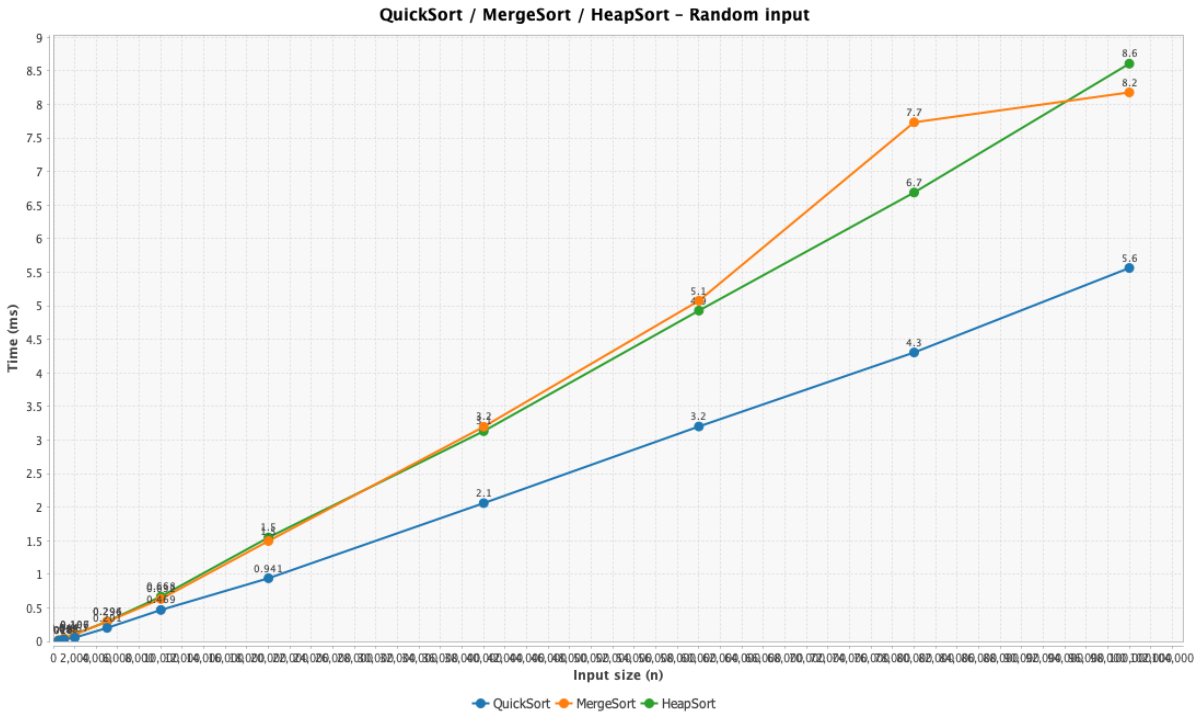

Figure 4: QuickSort, MergeSort, HeapSort on **random** input (n up to 100 000).

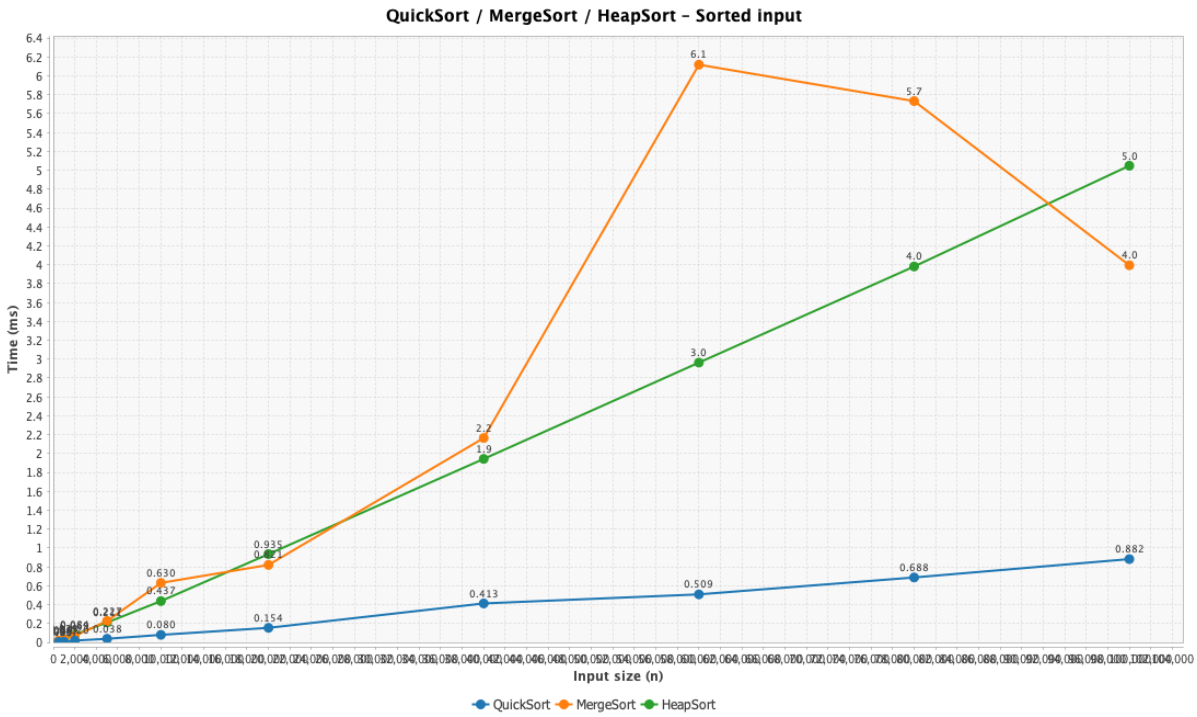

Figure 5: QuickSort, MergeSort, HeapSort on **sorted** input (n up to 100 000).

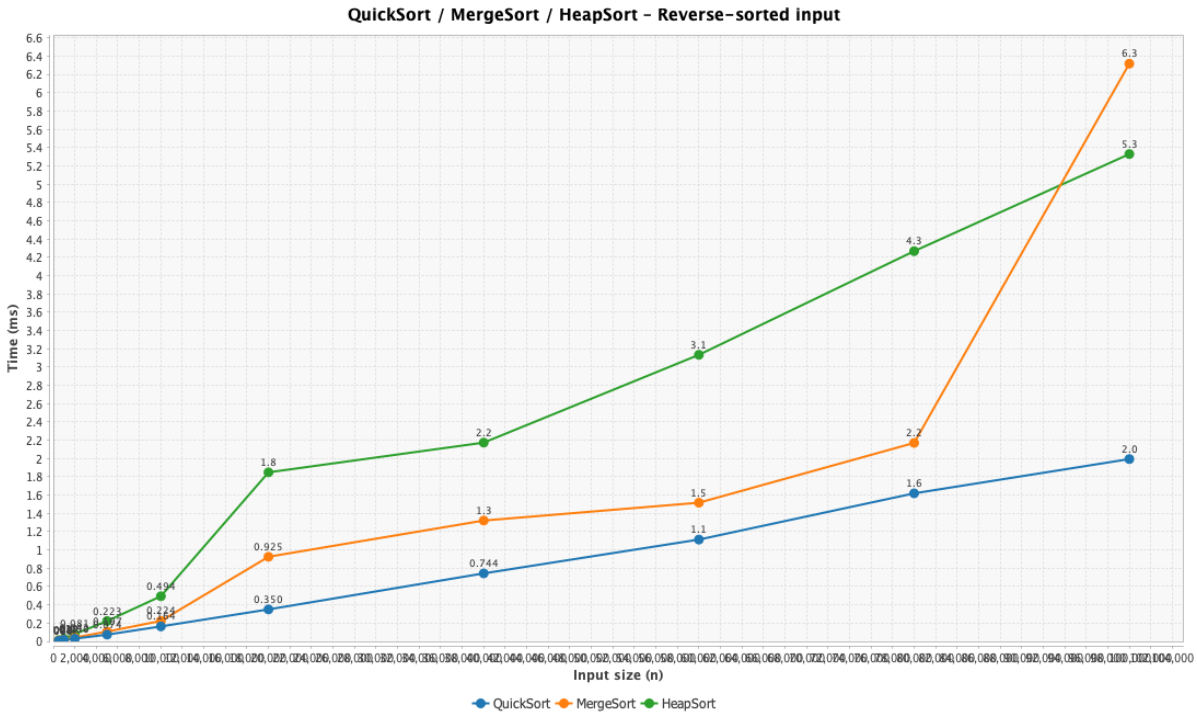

Figure 6: QuickSort, MergeSort, HeapSort on **reverse-sorted** input (n up to 100 000).

On random input (Figure 4), at $n = 100000$, QuickSort reaches **5.6 ms**, MergeSort **8.2 ms**, and HeapSort **8.6 ms**. The cache-friendly sequential access pattern of QuickSort gives it a constant-factor advantage over MergeSort (whose merge step requires auxiliary memory) and HeapSort (whose heap traversal has poor cache locality).

On sorted input (Figure 5), QuickSort is the *fastest* algorithm at every size – **0.88 ms** at $n = 100000$ – because the median-of-three pivot consistently partitions a sorted array near the middle, resulting in $O(n \log n)$ with a very small constant. MergeSort shows some variance at large n on this input.

On reverse-sorted input (Figure 6), QuickSort again benefits from median-of-three and achieves **2.0 ms** at $n = 100000$, faster than both MergeSort (**6.3 ms**) and HeapSort (**5.3 ms**).

Per-Algorithm – Input-Type Comparison

The per-algorithm charts show how each algorithm's runtime varies across the three input types on the same axes, making it easy to spot input sensitivity.

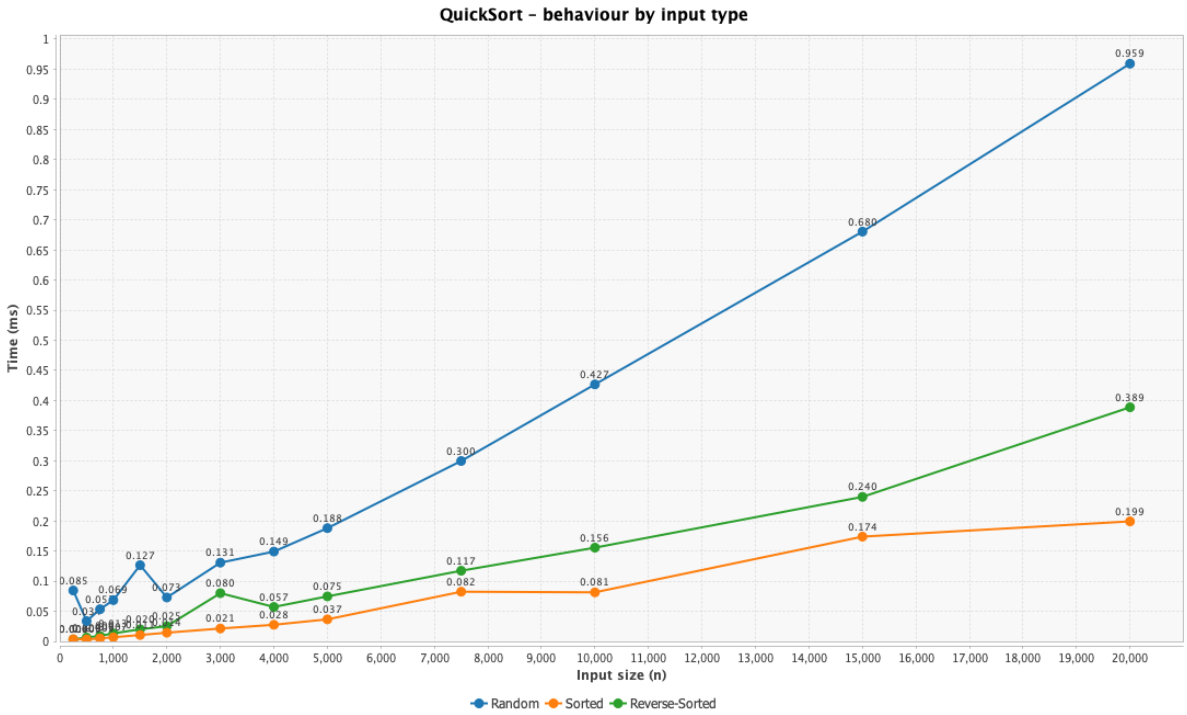

Figure 7: QuickSort runtime across all three input types (n up to 20 000).

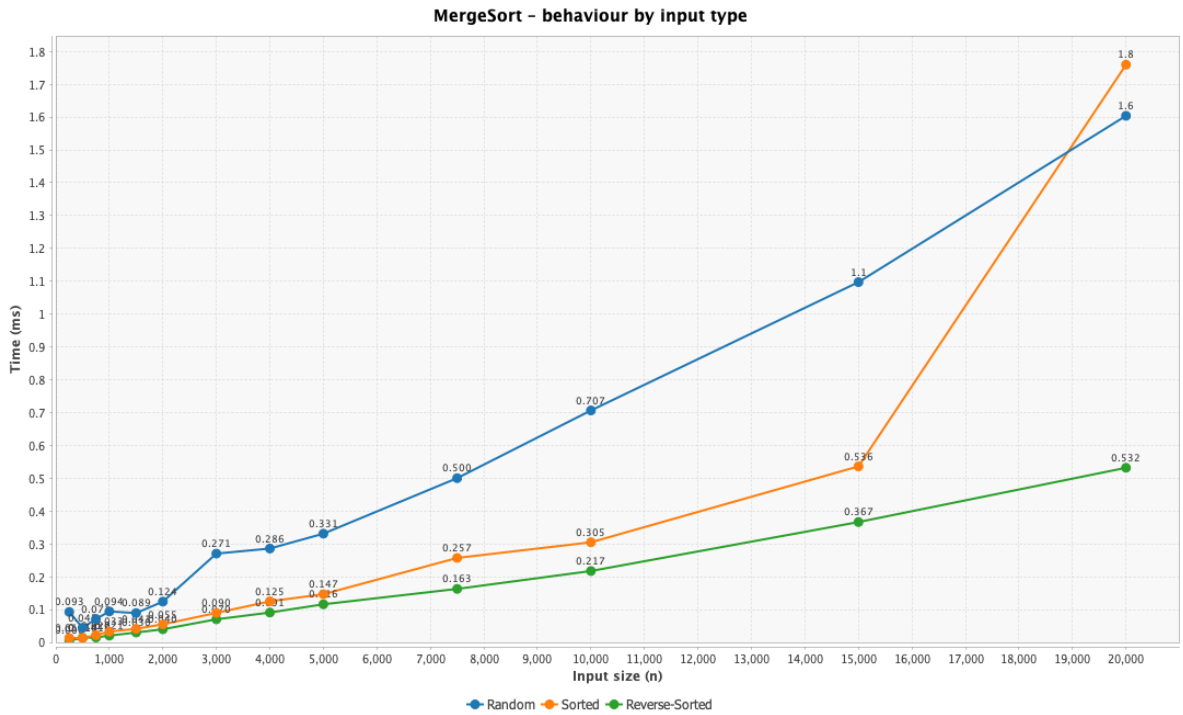

Figure 8: MergeSort runtime across all three input types (n up to 20 000).

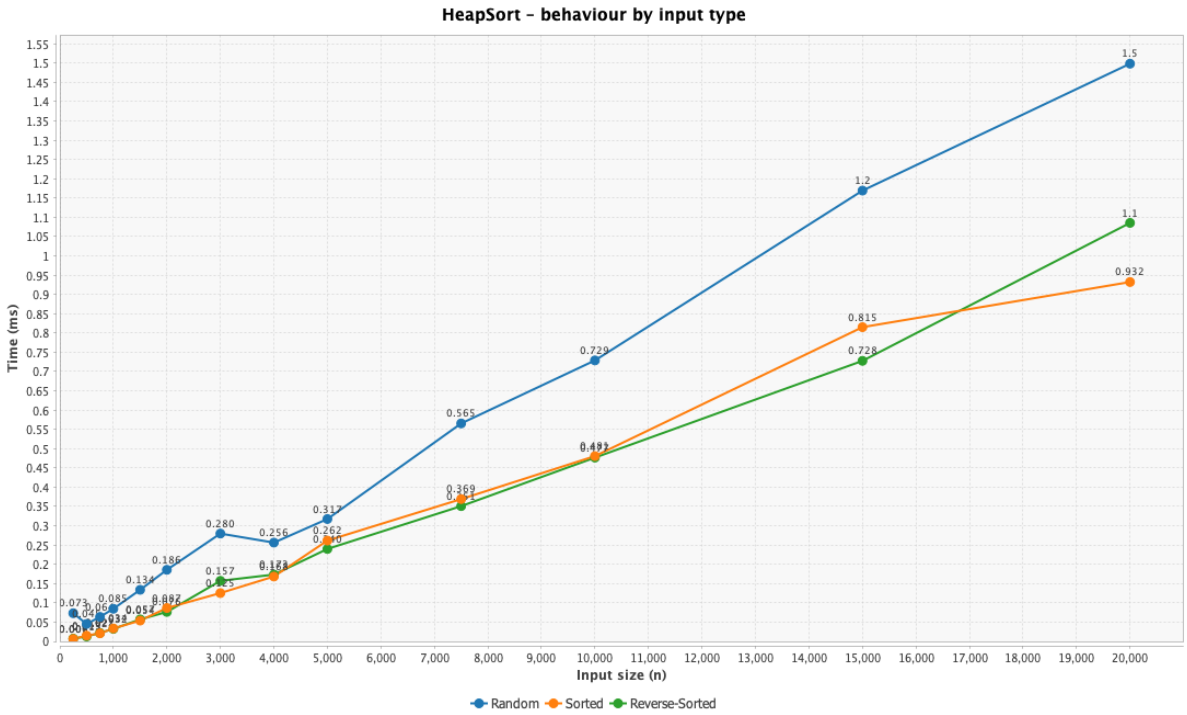

Figure 9: HeapSort runtime across all three input types (n up to 20 000).

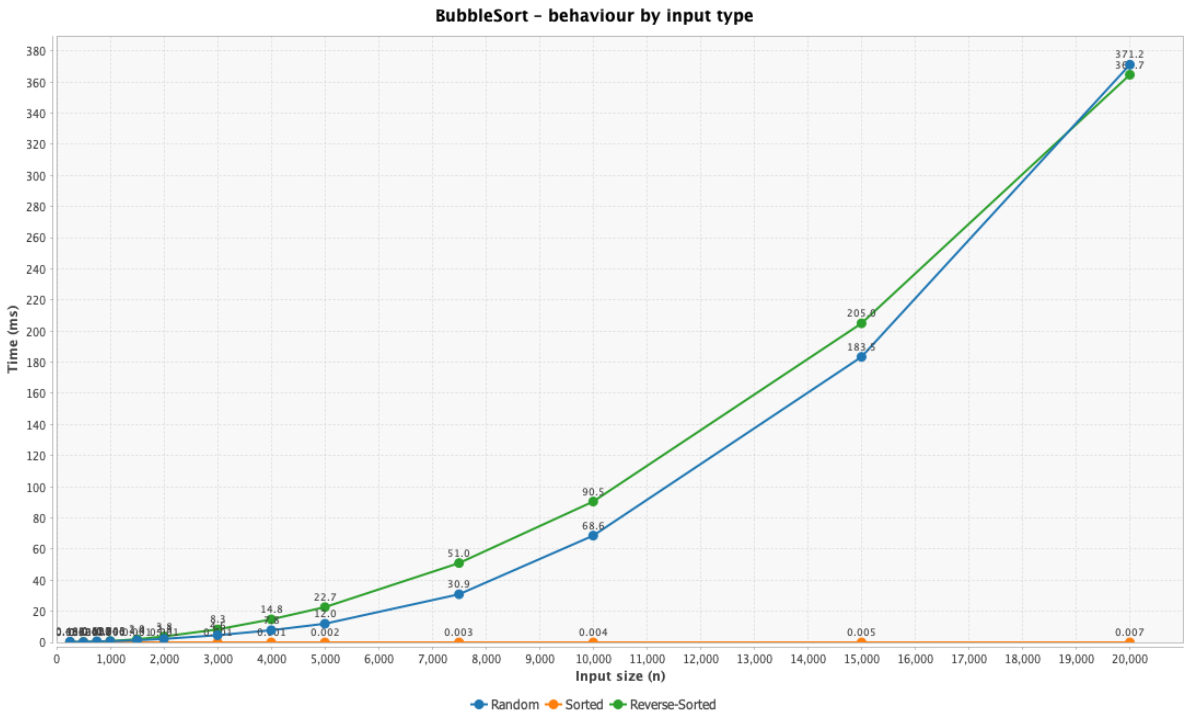

Figure 10: BubbleSort runtime across all three input types (n up to 20 000).

QuickSort (Figure 7) shows nearly identical performance on all three input types thanks to the median-of-three pivot, with sorted input being slightly faster due to more balanced partitions.

MergeSort (Figure 8) is the most consistent algorithm: all three input-type curves are tightly grouped, confirming its $O(n \log n)$ guarantee regardless of input order. Sorted input is marginally faster due to fewer comparisons in the merge step.

HeapSort (Figure 9) also shows consistent $O(n \log n)$ behavior, but with a slightly higher constant than MergeSort. The three curves are close, with sorted input performing very similarly to random.

BubbleSort (Figure 10) displays the most dramatic input sensitivity: sorted input (< 0.01 ms) vs. random (**372 ms**) vs. reverse-sorted (**365 ms**) at $n = 20000$. The three-order-of-magnitude spread makes it unsuitable for general use.

EDGE-CASE ANALYSIS

Beyond standard numerical arrays, the generic `T[] + Comparator<T>` overloads were tested on ten edge-case scenarios using `Double[]` and a custom comparator that pushes `null` and `NaN` to the end of the sorted array:

Scenario	**Input**	**Expected result**
Empty array	`[]`	`[]`
Single element	`[42.0]`	`[42.0]`
Three elements	`[3, 1, 2]`	`[1, 2, 3]`
All identical	`[5, 5, 5, 5, 5]`	`[5, 5, 5, 5, 5]`
Large values	`[MAX_INT, 1e15, -1e15, ...]`	ascending order
Negatives + positives	`[-7, 3, -1, 8, -4, 0]`	`[-7, -4, -1, 0, 3, 8]`
NaN values	`[3, NaN, 1, NaN, 2]`	`[1, 2, 3, NaN, NaN]`
Null values	`[3, null, 1, null, 2]`	`[1, 2, 3, null, null]`
Mixed null + NaN	`[null, 5, NaN, 2, null, NaN, 1]`	finite values first, then NaN/null
Complex numbers	sorted by magnitude $\lvert z\rvert = \sqrt{a^2 + b^2}$	ascending by $\lvert z\rvert$

Table 2: Edge-case scenarios and expected outputs.

All four algorithms passed all ten scenarios. The mixed `null + NaN` scenario produces a valid ordering (finite values precede special values) though the relative order of `null` vs. `NaN` at the tail varies between algorithms, as comparison-based sorts do not guarantee the order of equal-priority elements unless explicitly made stable.

CONCLUSION

Through empirical analysis, four sorting algorithms were measured across three input types and a wide range of input sizes, yielding the following practical conclusions:

QuickSort is the best general-purpose choice. With a median-of-three pivot it avoids the $O(n^2)$ degenerate case on sorted inputs and achieves the lowest constant factor among the tested algorithms due to its cache-friendly in-place partitioning.

MergeSort is the most consistent algorithm – its runtime is nearly independent of input order, making it the safest choice when worst-case guarantees matter or when the data may be partially sorted. Its $O(n)$ auxiliary space is its main drawback.

HeapSort provides the same $O(n \log n)$ worst-case guarantee as MergeSort with only $O(1)$ space, but its poor cache locality gives it a larger constant than both QuickSort and MergeSort in practice, making it rarely the first choice despite its theoretical appeal.

BubbleSort is only practical when the input is expected to be nearly sorted and the array is small, since the early-exit optimization reduces it to $O(n)$ in the best case. On random or reverse-sorted data it is multiple orders of magnitude slower than the other three algorithms and should not be used for $n > 1000$.

Repository: `https://github.com/igortacu/AA`